

# Monitoraggio, manutenzione e sicurezza per smart city: dalla gestione manuale ad un'architettura automatizzata basata su Zabbix

---

Laureando: Brando Calderara (Mat. 746352)

Relatore: Chiar.mo Prof. Francesco Gringoli

Anno Accademico 2025/2026



UNIVERSITÀ  
DEGLI STUDI  
DI BRESCIA

## Nome Sezione

---

## L'Infrastruttura (Secoval S.r.l.):

- Rete eterogenea distribuita sul territorio.
- Dispositivi IoT, Sicurezza e Networking.

## Classi di Sottoreti:

- TVCC (x.x.241.0/25) - Telecamere
- SW (x.x.241.128/25) - Switch
- AP (x.x.242.0/25) - Access Point
- LTE (x.x.242.128/25) - Connettività

## Il Problema (As-Is)

Totale assenza di uno strumento centralizzato per rilevare **guasti o manomissioni** in tempo reale.

## Obiettivo del Tirocinio (To-Be)

Progettare un'architettura di monitoraggio integrata, basata su mappa geografica (GIS) e connessa al sistema di ticketing aziendale.

## Il Core del Sistema: Zabbix e CMDB

- **La Piattaforma:** Adozione di **Zabbix** (Open-Source) per il monitoraggio IT tramite processi di *Auto-discovery* della rete.
- **Estrazione della Conoscenza (Data Ingestion):**
  - I metadati fisici (ID Palo, Coordinate GPS) risiedevano su archivi Excel statici.
  - Sviluppo di uno script **Python** (librerie pandas e openpyxl).
  - Utilizzo delle **API JSON-RPC** di Zabbix per associare automaticamente i metadati agli host scoperti sulla rete.

### Risultato: Integrazione GIS

Le coordinate iniettate via API hanno permesso di abilitare la **Geomap** nativa (su base OpenStreetMap). L'azienda può ora visualizzare istantaneamente lo stato e la posizione esatta di un guasto sul territorio.

# Rilevamento Guasti e Analisi Storica

## Logica di Rilevamento (Polling)

- Utilizzo del protocollo **ICMP** (Ping).
- *Business Rule*: Tolleranza di 1 ora prima di generare l'evento PROBLEM (evita falsi positivi per riavvii fisiologici).

## Il limite del Real-Time

- Al rientro del guasto (OK), la dashboard si pulisce, perdendo lo storico per le analisi direzionali.

## Soluzione: Data Logging e Reportistica

Un *Custom Script* intercetta ogni cambio di stato (PROBLEM/OK) e lo storicizza su un server Ubuntu. Questo abilita la generazione di:

- **Report Giornalieri**: Allarmi attivi, fluttuazioni di rete, focus su TVCC.
- **Report Mensili**: Uptime percentuale, classificazione host critici, analisi guasti per fascia oraria.

Oltre al guasto di rete, l'azienda necessitava di rilevare gli **oscuramenti fisici** delle telecamere (vernice, rami, atti vandalici).

- Le telecamere Hikvision dispongono di logiche Edge AI (Tamper Detection).
- **Gestione Configurazioni:** Script periodico in Polling per validare l'attivazione dei servizi XML a bordo camera.
- **Inversione del Paradigma:** Non più Polling, ma **Streaming asincrono**.

1. **Middleware Python:** Apre una connessione HTTP persistente (*Stream*) verso l'endpoint ISAPI delle telecamere.
2. **Parsing in Real-Time:** La telecamera invia frammenti XML solo quando si verifica un'anomalia. Lo script li intercetta e li analizza istantaneamente.
3. **Push verso Zabbix (Trapping):** I dati vengono iniettati in Zabbix tramite `zabbix_sender`, utility a riga di comando ad altissime prestazioni.
4. **Zabbix Trapper Items:** Item specifici nel template ricevono passivamente il dato, generando i trigger di allarme oscuramento.

L'ultimo step del progetto è l'interfacciamento con **Interacta**, la piattaforma cloud aziendale per la gestione dei task.

### Il Flusso (Custom Media Type):

Zabbix esegue uno script locale passando le "Macro" dell'evento:

- status (PROBLEM / RESOLVED)
- IP, ID\_palo, lat, lon
- problem\_ID

#### Logica di Business

Lo stato dell'evento (status) viene valutato dallo script Python per invocare le API REST di Interacta:

- **PROBLEM:** Apre un nuovo Ticket.
- **RESOLVED:** Chiude il Ticket associato.

## Sicurezza Server-to-Server: JWT e RS512

L'autenticazione verso Interacta è garantita dallo standard **JSON Web Token (JWT Assertion)**.

- **Firma asimmetrica (RS512):** Chiave privata locale (JSON), chiave pubblica sul cloud.
- **Libreria cryptography:** Struttura Header + Payload, firma RSA + Hash SHA512.

### Codice di firma (Python)

```
signature = private_key.sign(  
    signing_input, padding.PKCS1v15(), hashes.SHA512()  
)  
signed_jwt = f"{b64_header}.{b64_payload}.{b64url_encode(signature)}"
```